🪞 Autoevaluación

Tabla de contenidos

1 Fundamentos 1
2 Encapsulamiento 2
3 Herencia 3
4 Polimorfismo 3
5 Excepciones 4
6 Para pensar... 5

1 Fundamentos

1. ¿Qué diferencia hay entre una clase y un objeto? ¿Y entre un método y una función?
2. ¿Qué ocurre al ejecutar la siguiente definición? ¿Se crea algún objeto de tipo `Budin` en ese momento?

```
class Budin:
    def __init__(self):
        print("Creando budín")
```

3. ¿En qué momento se ejecuta `__init__`? ¿Cuál es su propósito? ¿Por qué no debería contener un `return`?
4. ¿Qué representa `self` dentro de un método? ¿Por qué no se debe pasar explícitamente al llamar a un método desde un objeto?
5. Considere la siguiente clase:

```
class Budin:
    def __init__(self, base):
        self.base = base
```

   - ¿Cuál es la diferencia entre `base` y `self.base`?
   - ¿Qué devuelve `Budin("vainilla").base`?
   - ¿Qué ocurriría si se intentara acceder a `Budin("vainilla").cobertura`?

6. ¿Por qué los dos objetos siguientes pueden tener valores distintos para el atributo `base`?

```
b1 = Budin("vainilla")
b2 = Budin("chocolate")
```

7. ¿Cuántos objetos crea el siguiente programa?

```
Budin("vainilla")
Budin("vainilla")
```

8. ¿Qué problema puede generar agregar un atributo solo a una instancia?

```
b1.cobertura = "chocolate"
```

   ¿Cómo podría evitarse usando un valor por defecto en `__init__`?

9. ¿En qué casos tiene sentido que un método devuelva `self`? De un ejemplo.
10. Analice el método `transferir` de `CuentaBancaria`. ¿Qué debería ocurrir si se intenta transferir más dinero del que hay disponible? ¿El valor que devuelve el método representa correctamente el resultado de la operación?

2 Encapsulamiento

1. ¿Qué es el encapsulamiento? Explíquelo usando la diferencia entre el interior y el exterior de un objeto.
2. ¿Por qué una función también puede considerarse una forma de encapsulamiento?
3. ¿Qué información debería poder obtener una persona que usa una clase sin necesidad de leer su implementación?
4. ¿Para qué sirven los *docstrings*? ¿Qué información conviene incluir en el *docstring* de un método?
5. ¿Qué muestra `help(CuentaBancaria)`? ¿En qué se diferencia de ejecutar `help(CuentaBancaria.depositar)`?
6. ¿Por qué no se obtiene un error al ejecutar el siguiente programa?

```
def saludar(nombre: str) -> str:
    return f"¡Hola, {nombre}!"
saludar(128)
```

7. ¿Qué son un *getter* y un *setter*? ¿Qué ventaja ofrece un *setter* frente a asignar directamente un atributo?
8. ¿Qué comunica por convención un atributo cuyo nombre comienza con _? ¿Python impide acceder o modificar ese atributo desde afuera?
9. ¿Qué diferencia hay entre `_nombre` y `__nombre` en una clase de Python?
10. Considere el siguiente código:

```
e = Estudiante("María")
e.__nombre = "Juana"
```

¿Por qué `e.getNombre()` puede seguir devolviendo `"María"`? ¿Cómo se llama el mecanismo involucrado?

11. ¿Cuándo se ejecuta el método decorado con `@nombre.setter`? ¿Qué sucede si el *setter* rechaza el nuevo valor?
12. ¿Cuál es la diferencia entre un atributo de instancia y un atributo de clase? ¿Qué tipo de información conviene guardar en cada uno?
13. ¿Qué diferencia hay entre un método de instancia y un método de clase? ¿Qué representa `cls` en un método decorado con `@classmethod`?
14. ¿Por qué `Estudiante.desde_texto("Ana, 2024")` puede considerarse un constructor alternativo?

3 Herencia

1. ¿Qué es la herencia? ¿A qué aspectos de la clase padre puede acceder la clase hija?
2. Si `c` es una instancia de `Cuadrado`, ¿a qué evalúan las siguientes expresiones? Justifique.

```python
class Rectangulo:
    pass

class Cuadrado(Rectangulo):
    pass

isinstance(c, Cuadrado)
isinstance(c, Rectangulo)
isinstance(c, object)
```

3. ¿Qué es la sobreescritura de métodos? ¿Cuándo tiene sentido usarla?
4. ¿Qué es una clase abstracta? ¿Y un método abstracto? ¿Cuándo conviene utilizarlos?
5. ¿Qué es la herencia múltiple?
6. ¿Qué sucede si una clase hija hereda de dos clases padres que implementan un método con el mismo nombre?
7. ¿Para qué sirve `super()`?

4 Polimorfismo

1. ¿Qué significa polimorfismo en el contexto de programación?
2. ¿Por qué el siguiente bucle no necesita preguntar si cada mascota es un `Perro`, un `Gato` o un `Pajaro`?

```python
for mascota in mascotas:
    mascota.hablar()
```

3. ¿Es necesario que dos clases compartan una misma clase padre para que puedan usarse polimórficamente? Piense en el ejemplo de rectángulos y círculos que ambos implementan `area`.
4. ¿Por qué se puede ordenar una lista que contiene rectángulos y círculos con esta expresión?

```
sorted(formas, key=lambda forma: forma.area())
```

5. ¿Qué método mágico se ejecuta al usar el operador `==`? ¿Por qué dos objetos `Rectangulo(3, 2)` distintos dan `False` si no se implementa ese método?
6. Relacione cada operación con el método mágico correspondiente:

```
a == b
a < b
a + b
a / b
```

7. ¿Qué diferencia hay entre `__repr__` y `__str__`? ¿Cuándo se utilizan automáticamente por Python? ¿Cómo podemos utilizarlos manualmente?
8. ¿Por qué la implementación de `__add__` de `Vector` devuelve un nuevo `Vector` en lugar de modificar alguno de los dos vectores originales?
9. ¿Qué significa "sobrecarga de operadores"?
10. Elija uno de los métodos `__len__`, `__contains__`, `__call__` o `__bool__`. ¿Qué operación de Python habilitaría al implementarlo en una clase?

5 Excepciones

1. ¿Qué indica un `NameError`? ¿Por qué un nombre de variable escrito incorrectamente puede producir esta excepción?
2. Sin ejecutar las expresiones, indique qué excepción produciría cada una y explique su causa.

```
5 + "5"

numeros = [10, 20, 30]
numeros[5]

numero = 42
numero.append(5)
```

3. ¿Qué función cumplen los bloques `try` y `except`? ¿Qué ocurre si dentro del `try` se lanza una excepción cuyo tipo no coincide con ninguno de los bloques `except`?
4. Considere el siguiente programa:

```python
try:
    numero = int(texto)
    resultado = 100 / numero
except ValueError:
    print("El texto no representa un número entero")
except ZeroDivisionError:
    print("No se puede dividir por cero")
else:
    print("Resultado:", resultado)
finally:
    print("Fin del intento")
```

¿Qué bloques se ejecutan y qué se imprime cuando `texto` vale `"diez"`, `"0"` y `"4"`?

5. ¿Cuándo se ejecuta un bloque `finally`? ¿Por qué resulta útil para liberar recursos incluso cuando una excepción no es capturada?
6. ¿Qué hace la instrucción `raise`? ¿Qué sucede con las instrucciones posteriores de la función cuando se ejecuta?
7. Al lanzar una excepción, ¿por qué suele ser preferible lanzar un `ValueError` específico en lugar de un `Exception` general?
8. ¿Qué efecto tiene ejecutar `raise` sin argumentos dentro de un bloque `except`? ¿En qué se diferencia de lanzar una excepción nueva?
9. ¿Para qué se utiliza `raise NuevaExcepcion(...) from error_original`? ¿Qué información conserva en el *traceback*?
10. ¿Por qué una excepción personalizada suele heredar de `Exception` o de alguna de sus subclases? ¿Qué ventaja ofrece crear una jerarquía de excepciones relacionadas?

6 Para pensar...

1. Sin ejecutar el programa, anote qué debería imprimir cada `print`. Luego ejecútelo y explique qué objetos cambiaron y cuáles no. Use `id` si lo considera útil.

```python
class ListaDeCompras:
    def __init__(self, productos):
        self.productos = productos

    def agregar(self, producto):
        self.productos.append(producto)
        return self

feria = ["naranjas"]
lunes = ListaDeCompras(feria)
martes = ListaDeCompras(feria)
```

```
lunes.agregar("pan")
print(lunes.productos)
print(martes.productos)
print(feria)
```

¿Es deseable el comportamiento del programa? Proponga un cambio en `__init__` para que cada objeto conserve su propia lista, sin impedir que se use una lista como argumento de entrada.

2. Una persona escribe esta clase para contar cuantas canciones agregó a cada lista:

```
class Playlist:
    canciones = []

    def agregar(self, cancion):
        self.canciones.append(cancion)

viaje = Playlist()
estudio = Playlist()

viaje.agregar("Justo que te vas")
estudio.agregar("Se vos")

print(viaje.canciones)
print(estudio.canciones)
```

Ejecútelo. ¿Qué error de diseño revela? Reescriba la clase para que las canciones sean un atributo de instancia y agregue, además, un atributo de clase que cuente cuántas `Playlist` se crearon.

3. Analice este intento de implementar un contador encadenable:

```
class Marcador:
    def __init__(self):
        self.puntos = 0

    def sumar(self, cantidad):
        self.puntos += cantidad

    def reiniciar(self):
        self.puntos = 0
        return self

resultado = Marcador().sumar(3).reiniciar().sumar(2)
```

¿En qué expresión aparece el primer error y por qué? Corrija el diseño para que la última línea sea válida.

4. Ejecute este programa con los valores `-1`, `0`, `3`, `True` y `2.5` como argumentos de `Boleto`. ¿Cuáles deberían aceptarse si el atributo representa una cantidad de viajes disponibles? Modifique el *setter* para hacer cumplir ese contrato y para que no se pueda llegar a un estado inválido después de crear el objeto.

```
class Boleto:
    def __init__(self, viajes):
        self.viajes = viajes

    @property
    def viajes(self):
        return self._viajes

    @viajes.setter
    def viajes(self, valor):
        self._viajes = valor
```

   ¿Qué condiciones debe cumplir `valor` para ser aceptado en el *setter*? ¿Qué mensaje daría en el caso de que no se cumplan? Justifique específicamente el caso de `True`, que en Python es una instancia de `int`.

5. La clase siguiente expone una propiedad de solo lectura, pero todavía permite modificar el estado interno desde afuera:

```
class Equipo:
    def __init__(self, integrantes):
        self._integrantes = integrantes

    @property
    def integrantes(self):
        return self._integrantes

equipo = Equipo(["Nora", "Lautaro"])
equipo.integrantes.pop()
print(equipo.integrantes)
```

   Explique por qué no intervino ningún *setter*. Cambie la implementación para que el código cliente pueda consultar los integrantes pero no alterar la colección interna por accidente. Verifique también qué ocurre si se modifica la lista original que se pasó al constructor.

6. Un sitio recibe una fecha de publicación en dos formatos. Complete el método de clase y decida cómo debería responder ante una entrada mal formada.

```
class Publicacion:
    def __init__(self, titulo, anio):
        self.titulo = titulo
        self.anio = anio
```

```
    @classmethod
    def desde_texto(cls, texto):
        # admite, por ejemplo, "La invención de Morel | 1940"
        pass
```

Luego compare `Publicacion.desde_texto(...)` con una función externa que construye y devuelve una `Publicacion`: ¿por qué el método de clase es una mejor decisión si después aparece la subclase `LibroDigital`?

7. Antes de ejecutar, prediga las tres salidas. Después cambie el orden de las clases base de `Registro` y vuelva a probar.

```
class ConMarcaDeTiempo:
    def descripcion(self):
        return "con fecha"

class ConAutor:
    def descripcion(self):
        return "con autor"

class Registro(ConMarcaDeTiempo, ConAutor):
    pass

r = Registro()
print(r.descripcion())
print(Registro.mro())
print(isinstance(r, ConAutor))
```

¿Qué regla usa Python para escoger el método?